

RamdiskGuardian — Z: 内存盘稳定性 / 备份 / 自愈方案

本仓库是这台机器上 **Primo Ramdisk (Z 盘)** 的稳定化方案：开机自启、镜像持久化、掉盘自愈、安全备份、出错告警。给本人和以后的 AI 维护者查阅。最后更新：2026-06-15。

🔧 **系统重装 / 换电脑**：直接看 **DEPLOY.md** (含 Primo 手动建盘 + **deploy.ps1** 一键部署)。

0. 一句话现状

Z: 是一块 **32GB 动态内存盘** (Primo Ramdisk 旗舰版 6.6.0, 驱动 **FancyRd**, 引导级)。它本身**易失** (内存盘, 断电即空)。我们用 **三层防线** 让它"稳定 + 内容不丢 + 出错可知"。

- ① 存在性 : Primo 「非临时盘」→ 每次开机由驱动自动重建 (需关掉 Windows 快速启动)
- ② 持久化 : Primo 镜像文件 `C:\PR-Image-Z.vdf` → 开机加载内容 / 关机保存内容
- ③ 兜底+自愈: 本仓库守护脚本 → 每 15 分钟安全备份; 掉盘后自动从备份还原; 出错弹窗告警

1. ⚠️ 关于「开机自启」的真相 (重要, 之前理解错过)

Primo Ramdisk 6.6.0 没有「随系统启动自动创建此磁盘」这个独立勾选框。 它的开机自启机制是：

- 在「虚拟硬盘参数」对话框 → 「属性」里，**不勾选「临时」** = 这是一块**持久盘** = 系统每次启动时由引导驱动自动重建。勾了「临时」才是一次性、重启不回来的盘。
- 所以正确做法就是：**建盘时别勾「临时」** (本机已是非临时盘 ☒)。

之前一直掉盘、配置写不进去的真凶 = Windows 快速启动 (Fast Startup)。 它让系统「假关机」(混合休眠 hiberboot)，不是真正冷启动 / 冷关机，于是 Primo 「关机保存配置、开机重建盘」的循环被破坏。

➡ **已处理**： **HiberbootEnabled** 已设为 **0** (关闭快速启动)。注册表位置：
`HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Power\HiberbootEnabled`

如何最终验证开机自启是否生效

- 正常**重启一次电脑** (关快速启动后是真正冷启动)。
- 开机后不做任何操作，看 **Z:** 是否**自动出现且为 32GB**。 - 出现 ☒ → 开机自启成功，三层防线齐活。 - 没出现 ☒ → 打开 Primo → 工具栏齿轮「设置」里找是否有「随 Windows 启动 / 开机加载」之类的**全局开关**需要打开；仍不行就回来排查 (守护脚本会在开机后 STATUS 标记 ERROR 并弹窗)。

2. 当前 Primo 盘配置 (本机实际值)

项目	值
盘符 / 文件系统 / 卷标	Z: / NTFS / RAMDISK
容量	32768 MB (32GB)
类型	SCSI 硬盘
内存模式	动态内存管理 + 紧凑模式 (按需占用, 写多少占多少)

项目	值
临时属性	未勾选 (= 持久盘 / 开机自启)
镜像	启用, 紧凑镜像 <code>C:\PR-Image-Z.vdf</code> (开机加载 / 关机保存)
NTFS 权限	Everyone: 修改 / 读取执行 / 写入

镜像建议: `C:\PR-Image-Z.vdf` 放在系统盘也能用; 若想挪到数据盘, 可改到 `E:\RamdiskImage\` (目录已建)。紧凑镜像只按实际用量增长, 不会一上来占 32GB。

关于「镜像持久化没有设置项」

Primo 里只要盘是非临时 + 勾了「启用镜像」, 默认行为就是 创建时载入镜像内容、退出/关机时保存内容——这就是持久化, 不需要额外开关。若想要「定时自动保存」(防断电丢更多), 在「虚拟硬盘参数」里点「镜像设置」按钮里找 定时/退出保存选项。即使没有它, 第 ③ 层守护脚本每 15 分钟也会把数据备份到 `E:\Z_Drive_Backup`。

3. 第 ③ 层：守护脚本（本仓库核心）

脚本: `zguardian.ps1` 调用链: 计划任务 `RAMDisk_Code_Backup` → `run_hidden.vbs` (隐藏窗口) → `sync_code.bat` → `zguardian.ps1` 触发: 用户登录时 + 之后每 15 分钟; 身份 10979 / 交互会话 (不是会话0) / 最高权限。

为什么是交互会话而不是会话0: 备份要在你登录、真正改动文件时跑才有意义; 而且 Z 是 全局盘 (实测 SYSTEM/会话0 也能看到), 交互会话同样看得到, 所以放交互会话最合适。

逻辑（何时从备份"回拉"补齐盘内容）

脚本只在**确有必要**时从备份回拉, 避免复活你删的文件: - 就绪标记 `Z:\.ramdisk_ready` 不存在 (全新盘 / 中途掉盘被重建) → 从备份还原。 - 每次开机后的首次运行 (盘刚从 Primo 镜像加载, 镜像可能比备份旧) → 按**新者为准**从备份 补齐更新的文件 (自动纠正"镜像滞后"——例如关机没存到最新镜像时)。 - 其余每次运行 → 只备份、不回拉 (你在 Z 上删的文件不会被复活)。

回拉只复制"备份比盘新"的文件 (`robocopy /E /XO`)。备份方向永远是**只增不减 + 新覆盖旧** (`/E /XO`, 无 `/PURGE`): 掉盘或加载到旧镜像都**不可能删除/缩小/降级备份**。代价: 你**主动删除的文件**会保留在 `E:\Z_Drive_Backup` 里 (为"绝不丢数据"做的取舍); 想清理直接手动删 `E:\Z_Drive_Backup` 即可。

安全护栏（吸取过的教训）

旧脚本用 `robocopy /MIR`, 某次掉盘后 `Z:\projects` 变空目录, `/MIR` 把"空"镜像到了 `E:\Z_Drive_Backup`, 把**唯一备份整盘删光** (这就是历史数据丢失的真因)。现脚本两道保险: ① 对每个通道三重判断 (Z 在? 源目录在? 源目录**递归含至少一个文件**? 任一不满足就跳过); ② 备份用 `/E /XO` (无 `/PURGE`), 从根上杜绝"删除/覆盖"类破坏——这是比旧 `/MIR` 更安全的设计。

出错怎么让你知道（健康告警）

每次运行写 `logs\STATUS.txt` (一行: 时间 + OK/WARN/ERROR + 详情): - **ERROR**: 开机 60 秒内 Z 还没出现 (掉盘/没自启)。 - **WARN**: Z 剩余空间 < 2GB (快满了)。 - 状态从 OK 变 WARN/ERROR 时, **弹一次 msg 弹窗**, 并追加 `logs>alerts.log`。

你随时双击 `E:\RamdiskGuardian\logs\STATUS.txt` 就能看最新健康状态。

4. 目录结构约定

```
Z:\
  projects\    ← Java/Python 等项目（贵重，纳入备份）
  docs\        ← 文档（贵重，纳入备份）
  others\      ← 其他重要文件（贵重，纳入备份）
  Caches\      ← 易失缓存（永不备份，掉盘自动重建）
    ChromeCache\    ← Chrome HTTP 缓存 junction 目标
    360zip_temp\    ← 360 压缩解压临时目录
  TEMP\        ← 临时
  .ramdisk_ready ← 守护脚本的就绪标记（隐藏）
```

约定：`projects/docs/others` = 备份区；`Caches/*` = 易失区（不备份）。

5. 四个功能的状态

功能	状态	说明
① Chrome HTTP 缓存	🕒 已配置/收益有限	junction <code>...\Default\Cache → Z:\Caches\ChromeCache</code> 有效且可写、Chrome 已重启接好。但 实测 Chrome 149 这次没往该目录写 （HTTP 响应多走内存缓存）；真正在频繁读写的 <code>Default\Code Cache</code> 仍在 C 盘 。要让内存盘真正给 Chrome 提速，建议照 §8 把 <code>Code Cache</code> 、 <code>GPUCache</code> 也搬到 Z（那才是热点）。
② 360 压缩缓存	✅ 正常	<code>360zip_config.ini</code> 的 <code>ExtractTmpDir = Z:\Caches\360zip_temp</code> ，目录已就绪。
③ Java/Python	🕒 按既定取舍	项目放 <code>Z:\projects</code> （快 I/O）；依赖缓存（.m2/pip）按你的选择 留在 C 盘 （持久，掉盘不用重下）。完整加速规划见 §7。
④ 重要文件目录	✅ 正常	<code>Z:\others</code> 、 <code>Z:\docs</code> 。

6. 内存挤占会不会掉盘？（结论 + 对策）

结论：内存挤占本身不会掉盘，最多临时写入报错；内存一恢复就自动正常。

- 盘写满 32GB → 报“磁盘已满”、写入失败，但盘在、已有文件不丢。
- 别的程序吃光系统内存、盘要不到内存 → 这次写入失败报错，但盘和已有数据都在（已占内存是非分页、驱动锁定的）；内存松了写入恢复。
- 真正“掉盘+数据没”的是：**断电/蓝屏/硬重启**（→ 第③层开机自动从备份还原）、**没开机自启**（→ §1 已解决）。

你选了“保持动态内存”：不会因内存压力掉盘，只占实际用量。（若哪天想要“连写入失败都不要”的绝对空间保证，可在 Primo 改成「固定内存」预留满 32GB——代价是永久占用 32GB 物理内存；当前 64GB 也扛得住，但非必要。）

“出错我要知道”：已由 §3 的 STATUS / alerts / 弹窗实现。

7. Java / Python 提速规划（IDEA + Maven + DataGrip）

你的取舍是 **项目放 Z、依赖缓存留 C**。在此前提下的最佳实践：

放 Z (快、可再生, 掉盘自动重建/重下)

- **项目源码 + 编译产物**: 把活跃项目放 `Z:\projects\<工程>`。 `target/`、`build/`、`out/` 在内存盘上读写极快, 打包/部署明显提速。
- **IntelliJ IDEA 索引/系统目录** (可选, 提速最明显): 编辑 `IDEA安装目录\bin\idea.properties` (或 `Help → Edit Custom Properties`): `idea.system.path=Z:/Caches/idea-system` `idea.log.path=Z:/Caches/idea-log` 索引在内存盘上极快; 掉盘后 IDEA 自动重建索引, 不影响代码安全。
- **DataGrip 同理** (可选): `datagrip.system.path=Z:/Caches/datagrip-system`。

留 C (持久, 避免掉盘后重新下载)

- **Maven 本地仓库** `~/.m2`: 保持在 C 盘 (你的选择)。这样掉盘不用重下依赖。
- 若哪天想极致提速、愿意承担掉盘后首次构建重下: 在 `~/.m2/settings.xml` 设 `<localRepository>Z:\Caches\maven-repo</localRepository>`, 并在 §4 的“不备份”区。
- **pip / npm 缓存**: 同理, 默认留 C。

落地清单 (要做时再做, 需关相关程序)

1. 在 `Z:\projects` 下放/迁工程。
2. 改 `idea.properties` 的 `idea.system.path` 指向 `Z:/Caches/idea-system` (先建目录)。
3. 守护脚本已自动把 `Z:\Caches\*` 当易失区, 无需手工纳管。

8. 可选增强项

- **Chrome 缓存完整搬到 Z**: 关闭 Chrome 后, 把 `Default\Code Cache`、`Default\GPUCache` 也做成指向 `Z:\Caches\` 的 junction (命令示例): `powershell # 关闭 Chrome 后执行 (先建目标目录, 再做 junction)`
`$base="C:\Users\10979\AppData\Local\Google\Chrome\User Data\Default" New-Item -ItemType Directory -Force "Z:\Caches\ChromeCodeCache" | Out-Null cmd /c rmdir /s /q "$base\Code Cache" cmd /c mklink /J "$base\Code Cache" "Z:\Caches\ChromeCodeCache"` 注意: 做了这个之后, 请把 `Z:\Caches\ChromeCodeCache` 加进 `zguardian.ps1` 的 `$dirs` 列表, 这样掉盘后守护脚本会自动重建它, junction 才不会再悬空。
- **镜像挪到数据盘**: 把 Primo 镜像从 `C:\PR-Image-Z.vdf` 改到 `E:\RamdiskImage\Z.vdf`。
- **定时保存镜像**: Primo「镜像设置」里开启定时保存, 缩小断电丢数据窗口。

9. 文件清单 (本仓库)

```
RamdiskGuardian/
├─ README.md          本文件（原理与运维）
├─ README.pdf         本文件导出的 PDF（自动同步到 GitHub）
├─ DEPLOY.md          快速部署指南（系统重装 / 换电脑）
├─ DEPLOY.pdf         快速部署指南的 PDF（自动同步到 GitHub）
├─ deploy.ps1         一键部署（关快速启动/建目录/注册任务/Chrome junction，幂等）
├─ zguardian.ps1      守护脚本（备份 + 掉盘自愈 + 健康告警；路径可移植）
├─ sync_code.bat      入口（被 VBS 调用，%~dp0 转调 zguardian.ps1）
├─ run_hidden.vbs     隐藏窗口启动器（被计划任务调用，自定位）
├─ ramdrive.txt       可选：非 Z 盘符时由 deploy.ps1 写入
├─ .gitignore         忽略 logs/ 等运行时产物
├─ archive/
│   └─ sync_code.bat.bak_20260614 原始备份脚本（含危险 /MIR，仅留档）
├─ docs/
│   └─ primo_setup.png  Primo 界面标注图
└─ logs/              运行时日志（不入库）
    ├─ guardian.log   └─ STATUS.txt └─ alerts.log
```

相关但**不在本仓库**（留在原位，各有其任务/用途）： - 备份目标： `E:\Z_Drive_Backup\{projects,docs,others}` - 桌面其他脚本： `auto_backup.ps1` (H 盘软件清单备份任务)、 `检查运行状态.vbs` 等，与本项目无关，未动。

10. 运维速查

```
# 手动跑一次守护（重建骨架/备份/还原）
powershell -NoProfile -ExecutionPolicy Bypass -File E:\RamdiskGuardian\zguardian.ps1

# 看健康状态 / 日志
Get-Content E:\RamdiskGuardian\logs\STATUS.txt
Get-Content E:\RamdiskGuardian\logs\guardian.log -Tail 20

# 看/改备份频率（计划任务）
Get-ScheduledTask RAMDisk_Code_Backup | % { $_.Triggers }

# 改容量/内存模式/镜像：在 Primo 里删盘重建（盘空时零风险），按 §1/§2 设置
```

11. 变更历史 & 已知数据丢失

- **2026-06-15:**
- 升级 `build_docs_pdf.py` 支持指定工作目录与自动推送到 GitHub，生成并同步了本项目的 PDF 文档。
- 优化 `deploy.ps1` 中的 Chrome 缓存 Junction 重建逻辑，使其在内存盘符发生变更后重新运行部署时，能自动检测差异并安全更新 Junction 指向。
- **2026-06-14:** 发现并修复掉盘问题；重写安全备份脚本；加开机自愈+健康告警；关闭 Windows 快速启动；盘从 20GB→32GB、加镜像持久化（非临时盘）。

- **已知丢失：**本次接手前，`Z:\projects/docs/others` 原始数据已丢失——旧 `/MIR` 脚本 在更早一次掉盘时把 `E:\Z_Drive_Backup` 清空了，本地无其他副本。若那些是 git工程， 可从 GitHub 远端找回。